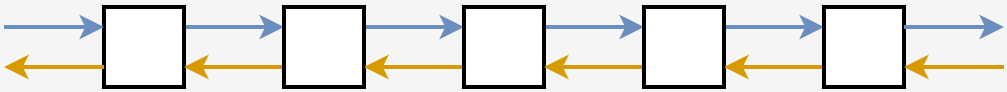
Memory Corruption

Taller de Seguridad

Ing. Martin Di Paola

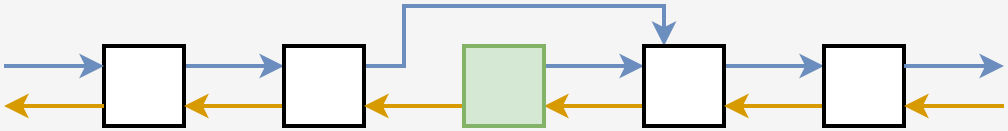


double
linked list



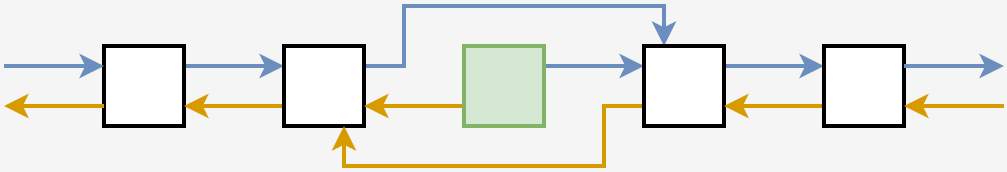
double
linked list

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



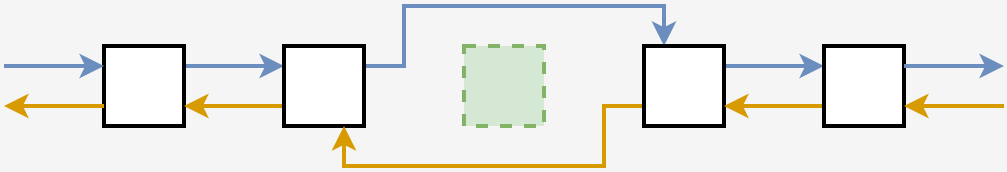
double
linked list

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



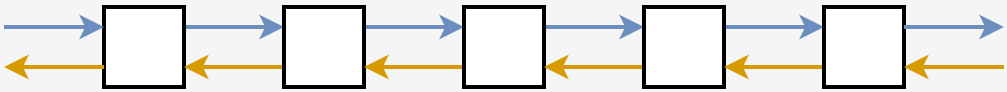
double
linked list

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



double
linked list

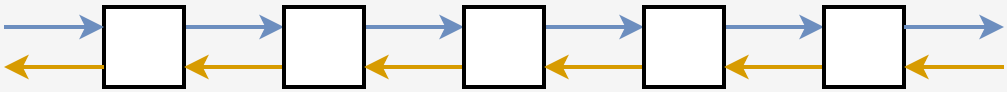
```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



double
linked list

what do
you see?

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

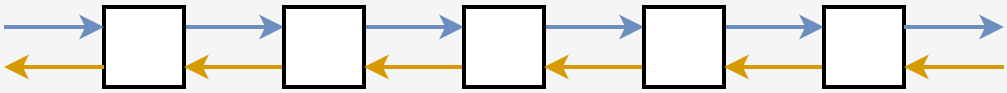


double
linked list

what do
you see?

a remove-node-
from-list primitive?

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



double
linked list

what do
you see?

a remove-node-
from-list primitive?

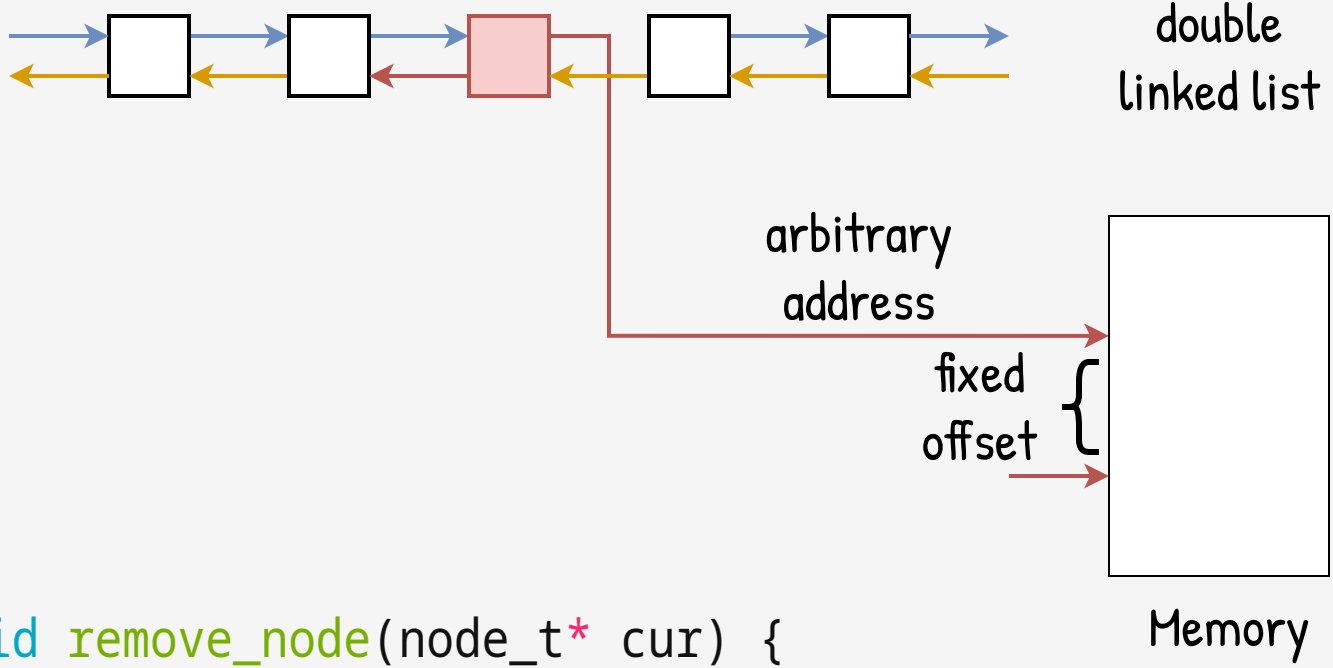
an arbitrary write
primitive!

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

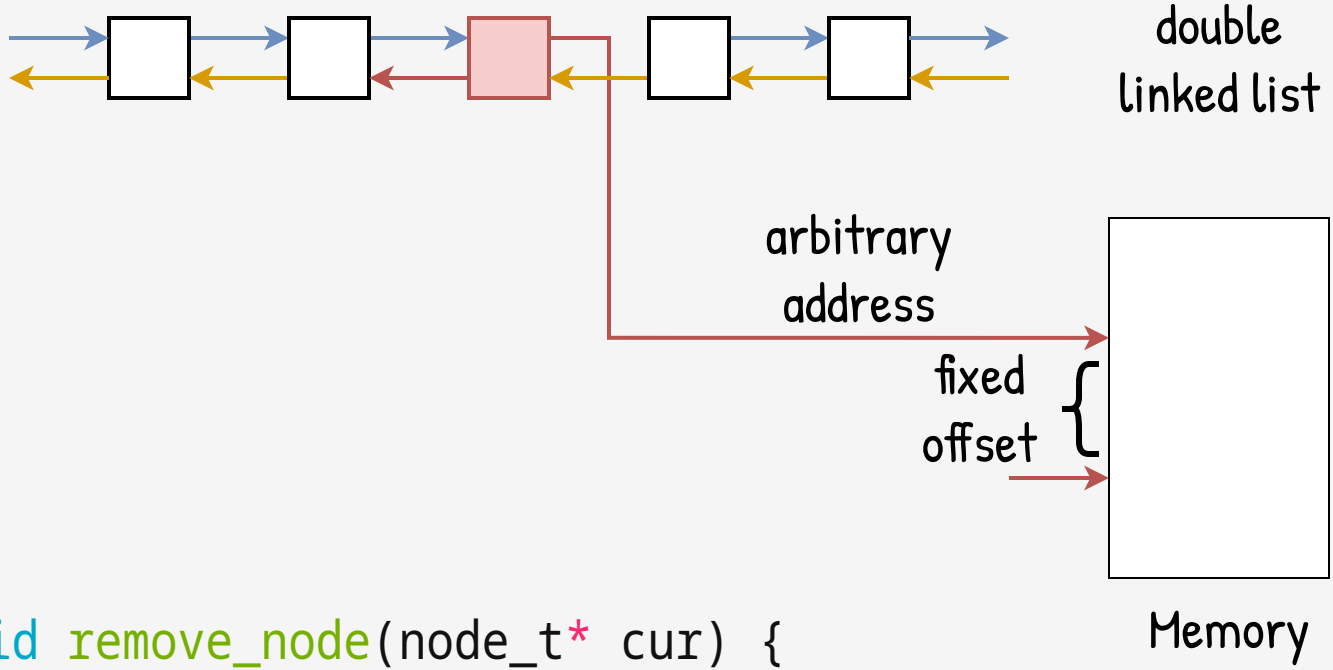


double
linked list

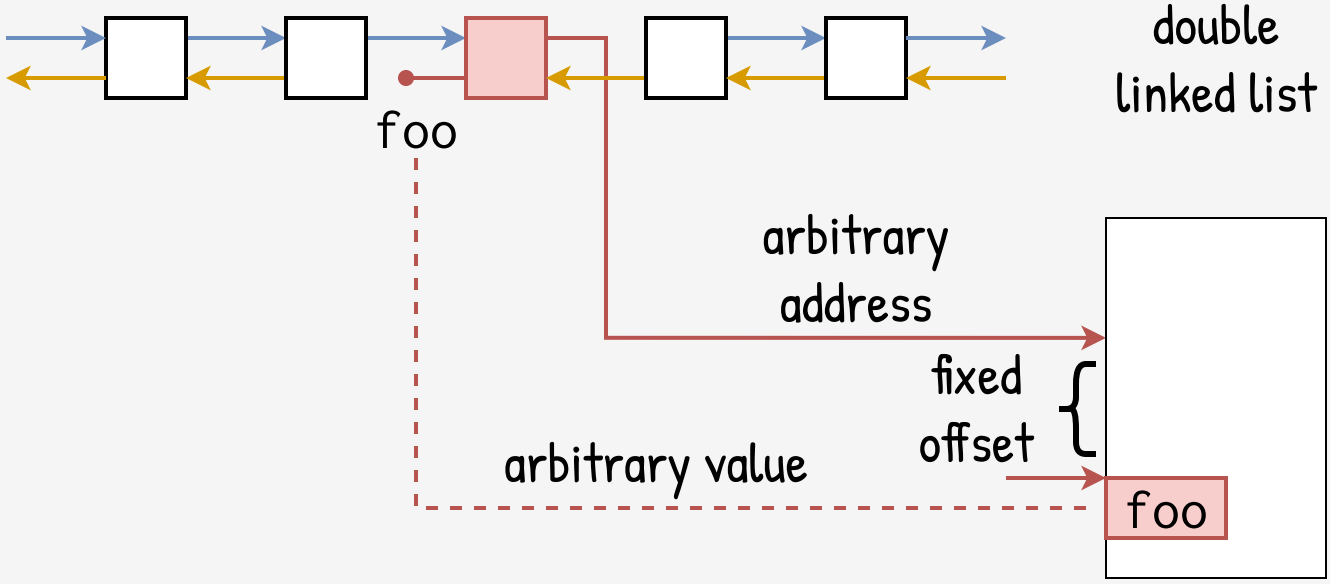
```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



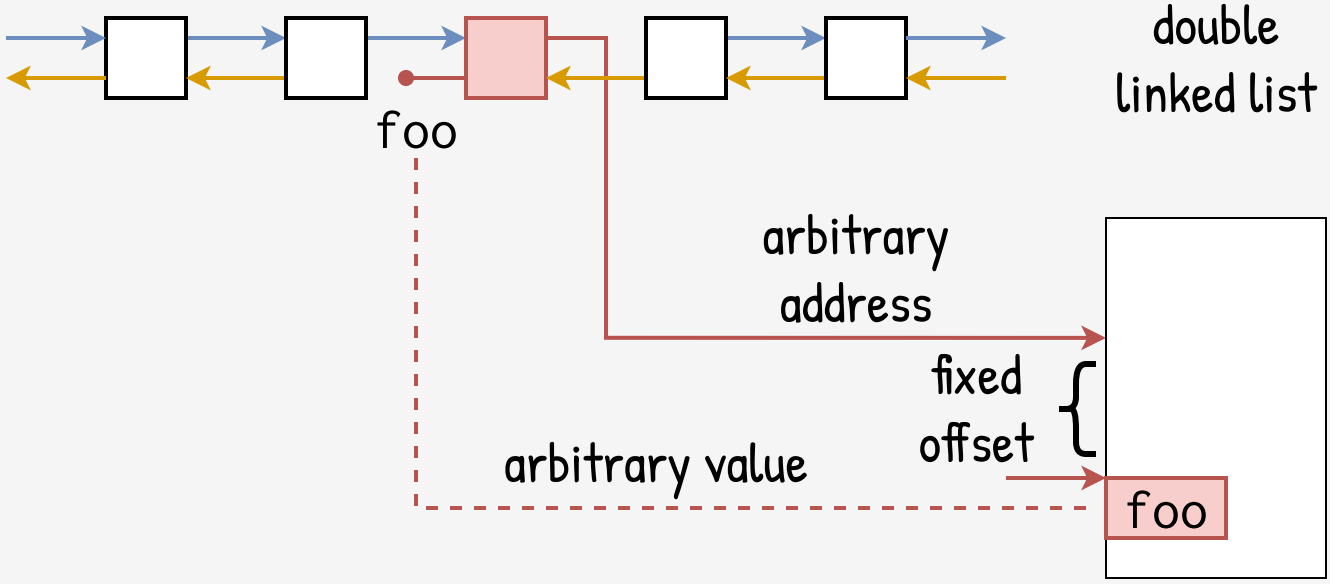

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

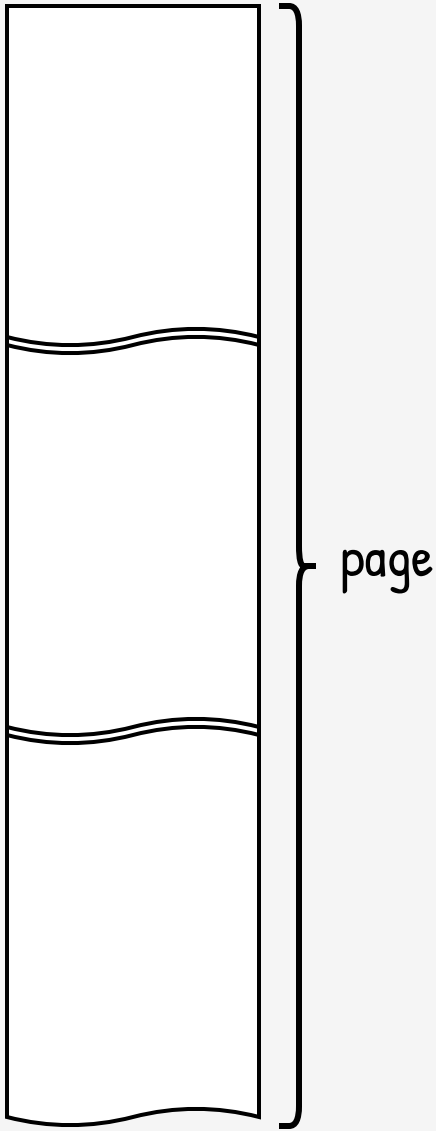



```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

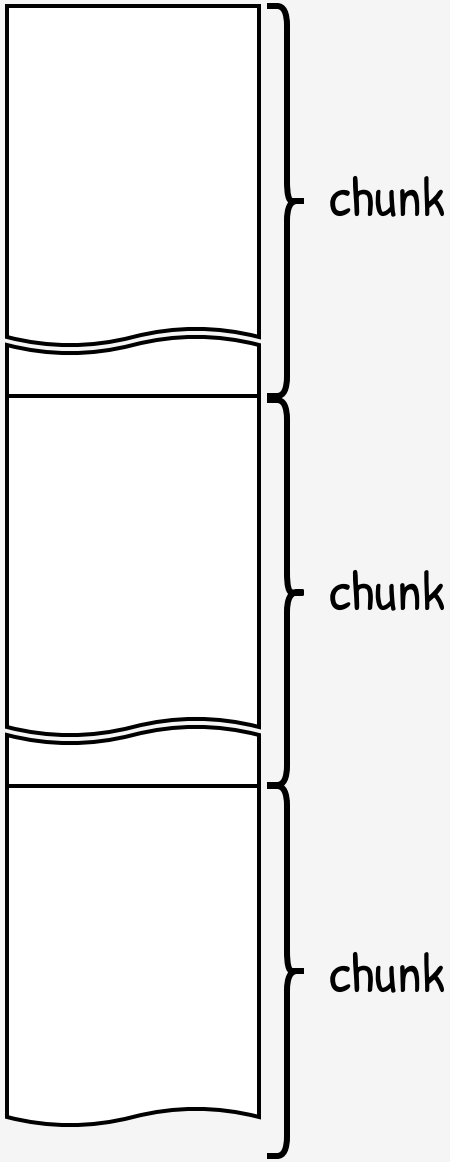
unwanted
side effect

```
malloc(64)
```

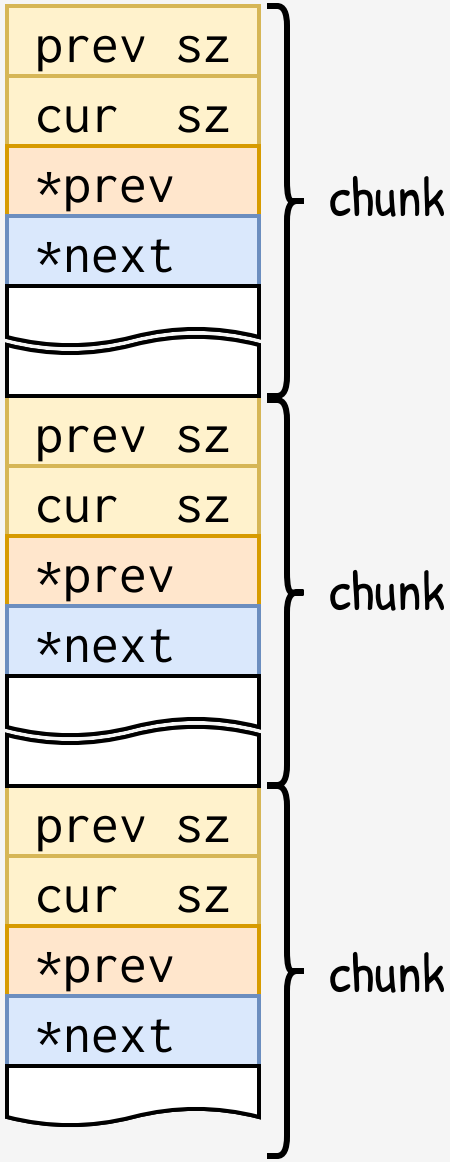
malloc(64)



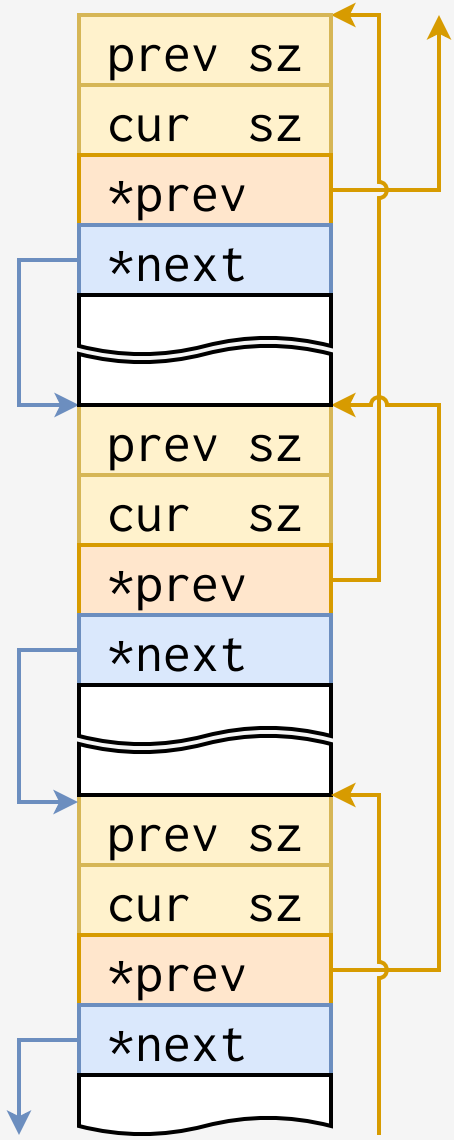
malloc(64)



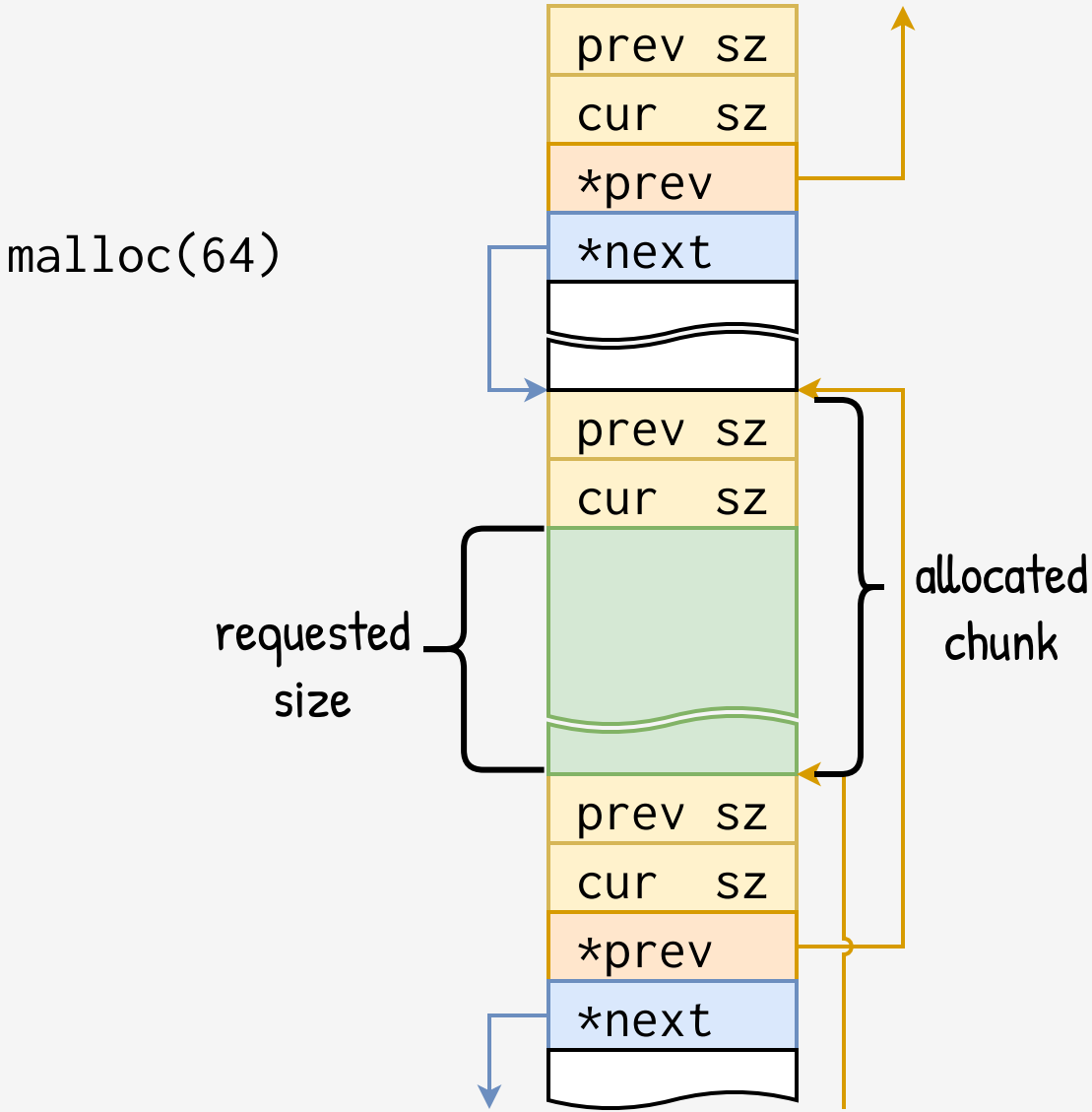
malloc(64)

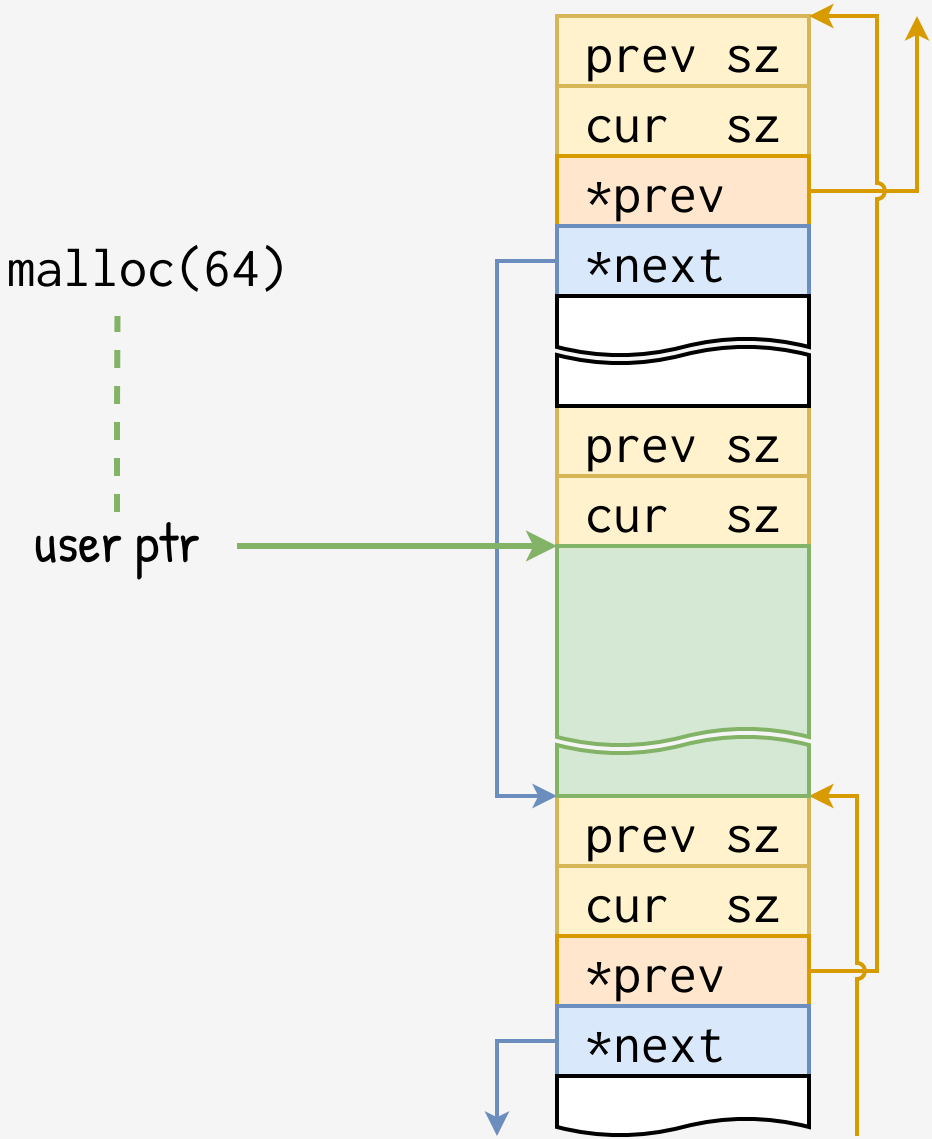

prev sz
cur sz
*prev
*next
chunk
prev sz
cur sz
*prev
*next
chunk
prev sz
cur sz
*prev
*next
chunk

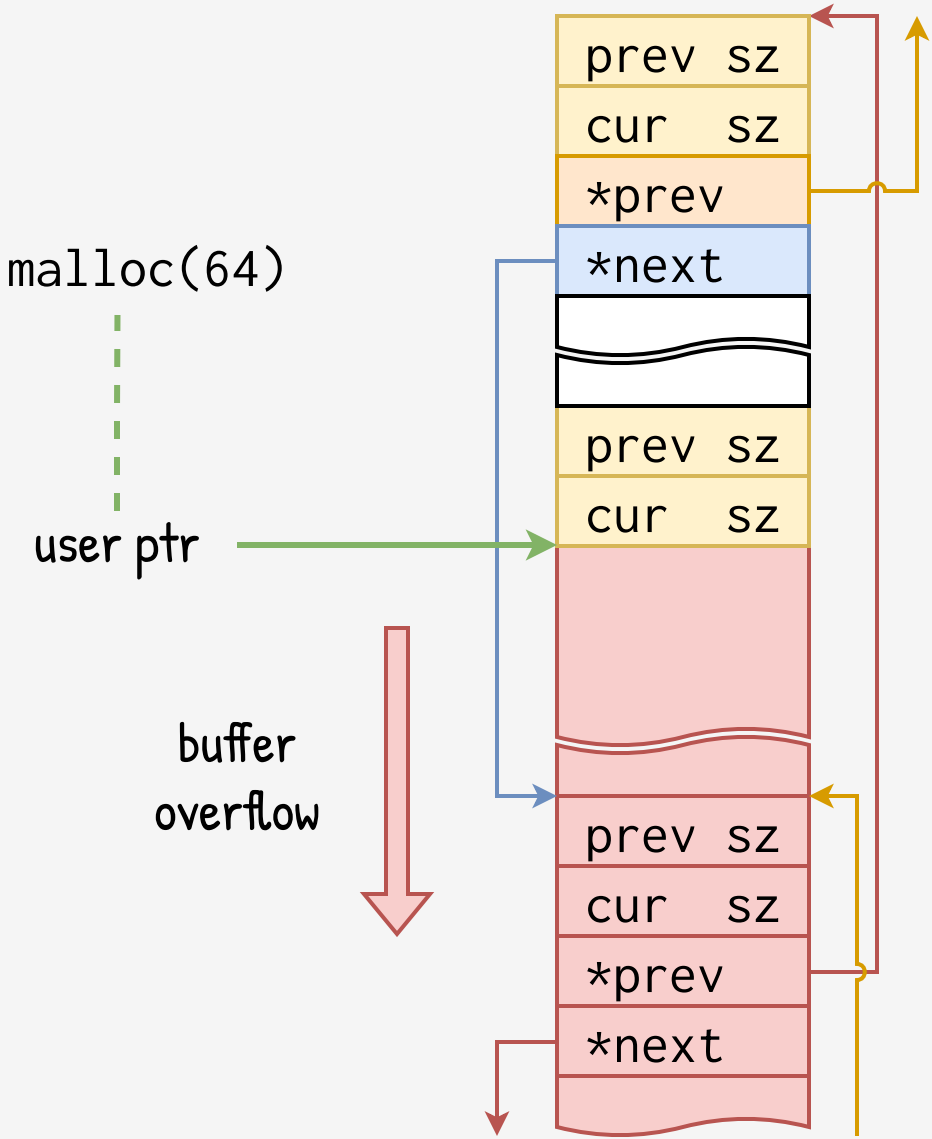
malloc(64)

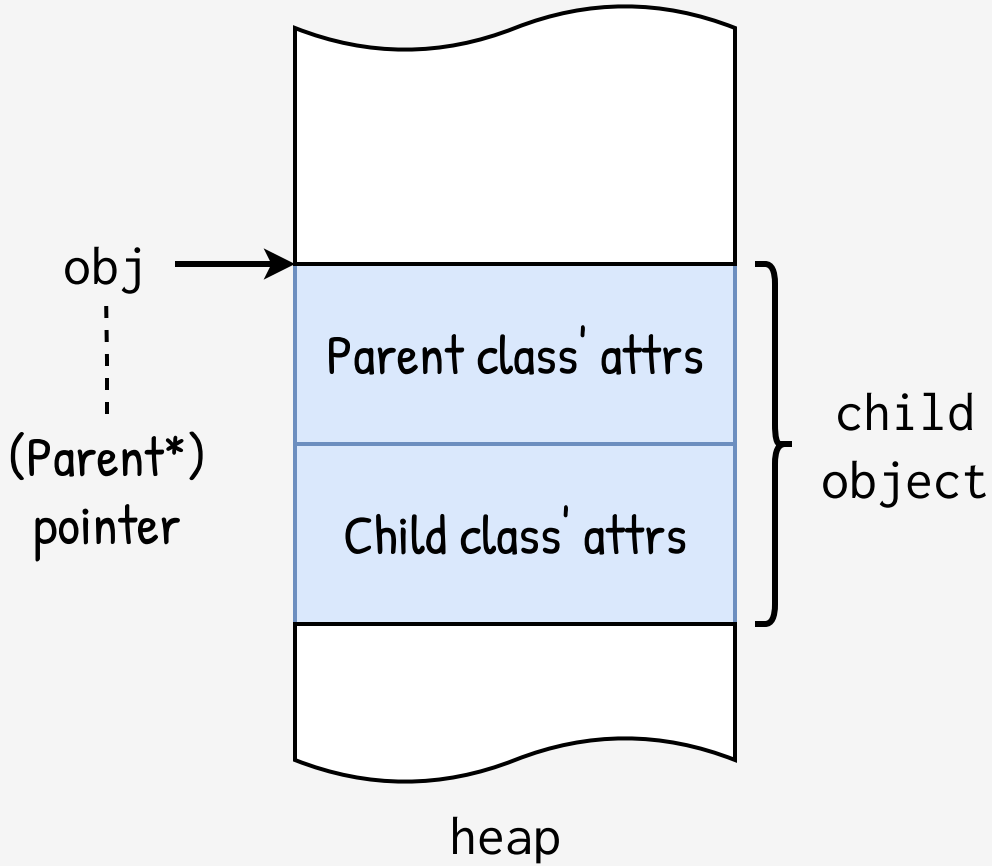


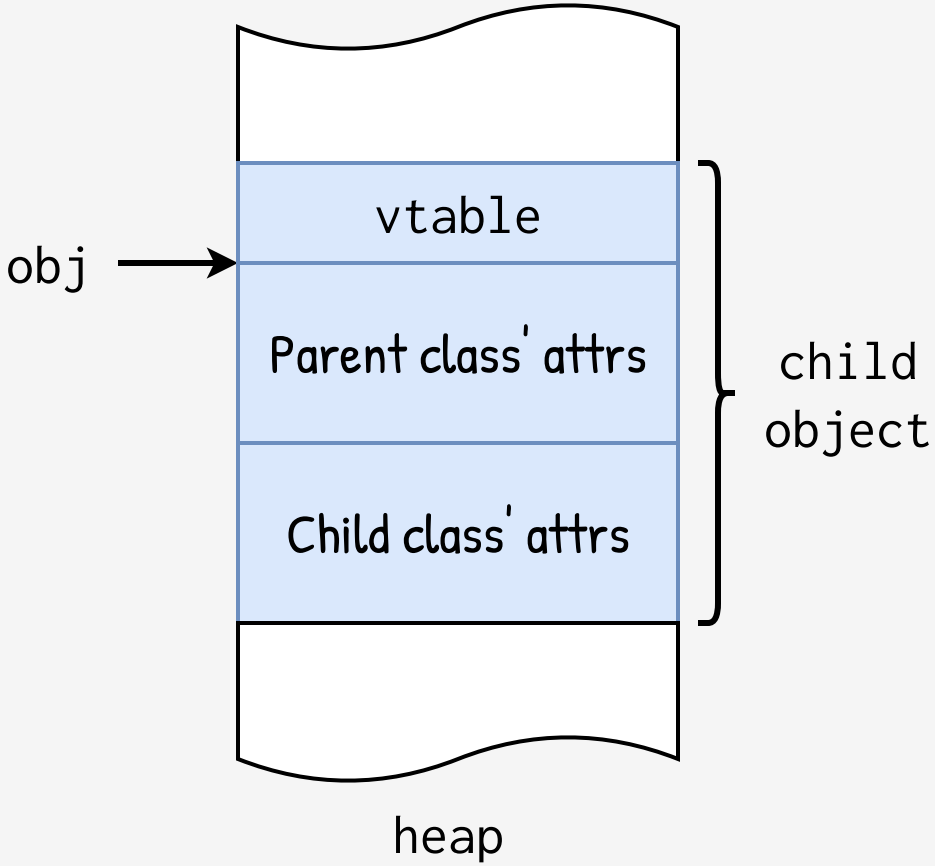
double linked list of ***free*** chunks

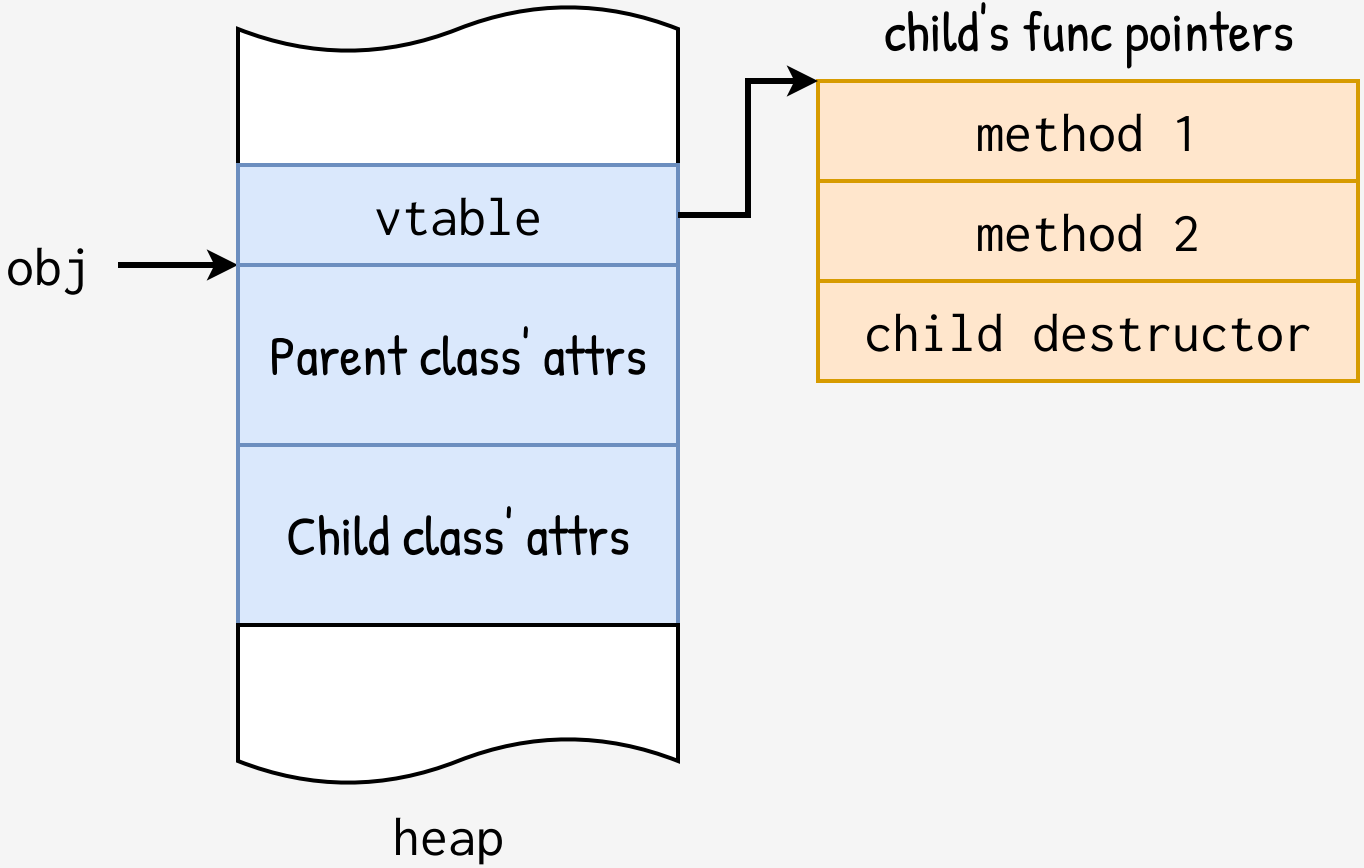

malloc(64)
prev sz
cur sz
*prev
*next
prev sz
cur sz
requested size
allocated chunk
prev sz
cur sz
*prev
*next

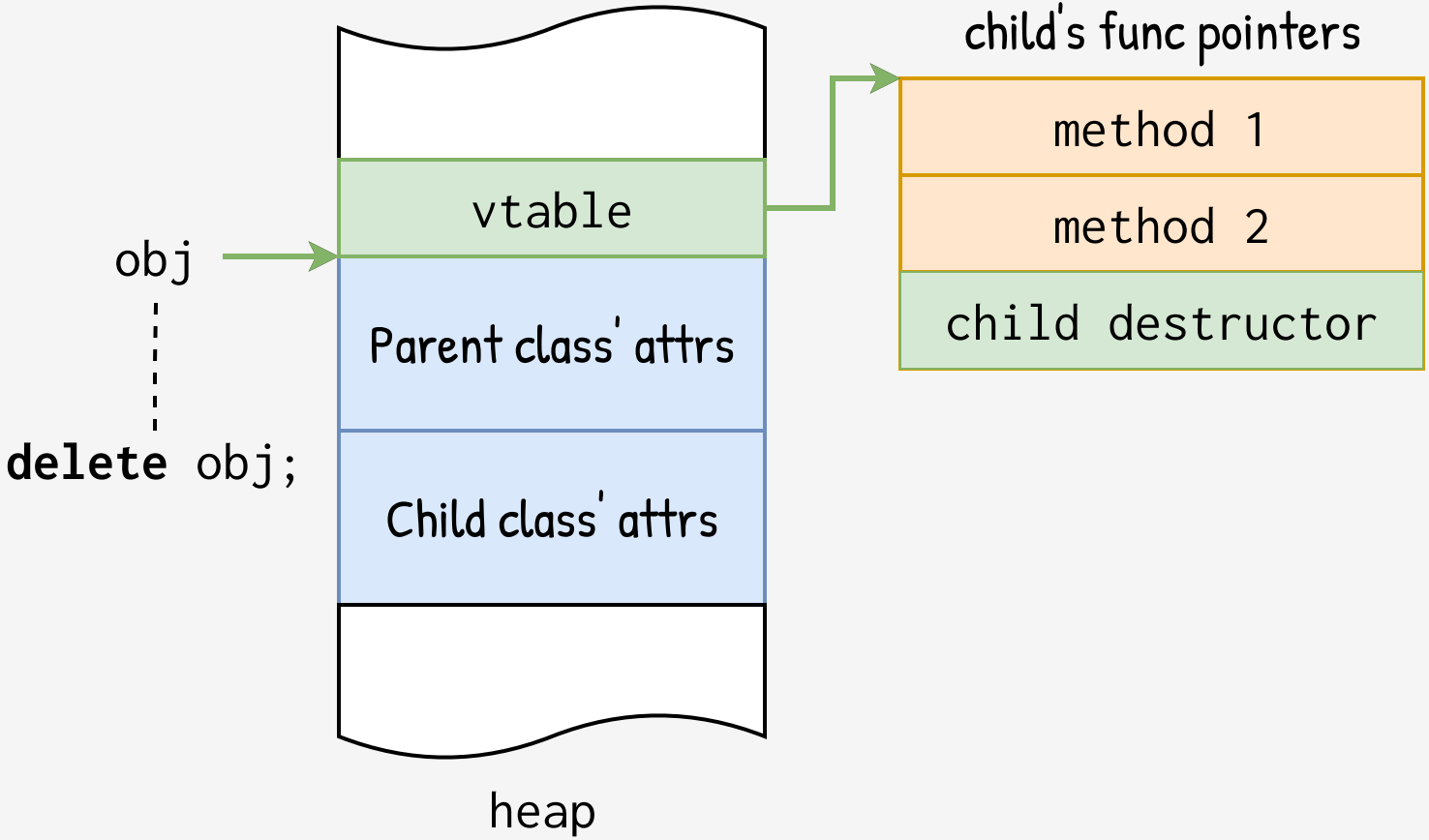

malloc(64)
user ptr
prev sz
cur sz
*prev
*next
prev sz
cur sz
prev sz
cur sz
*prev
*next

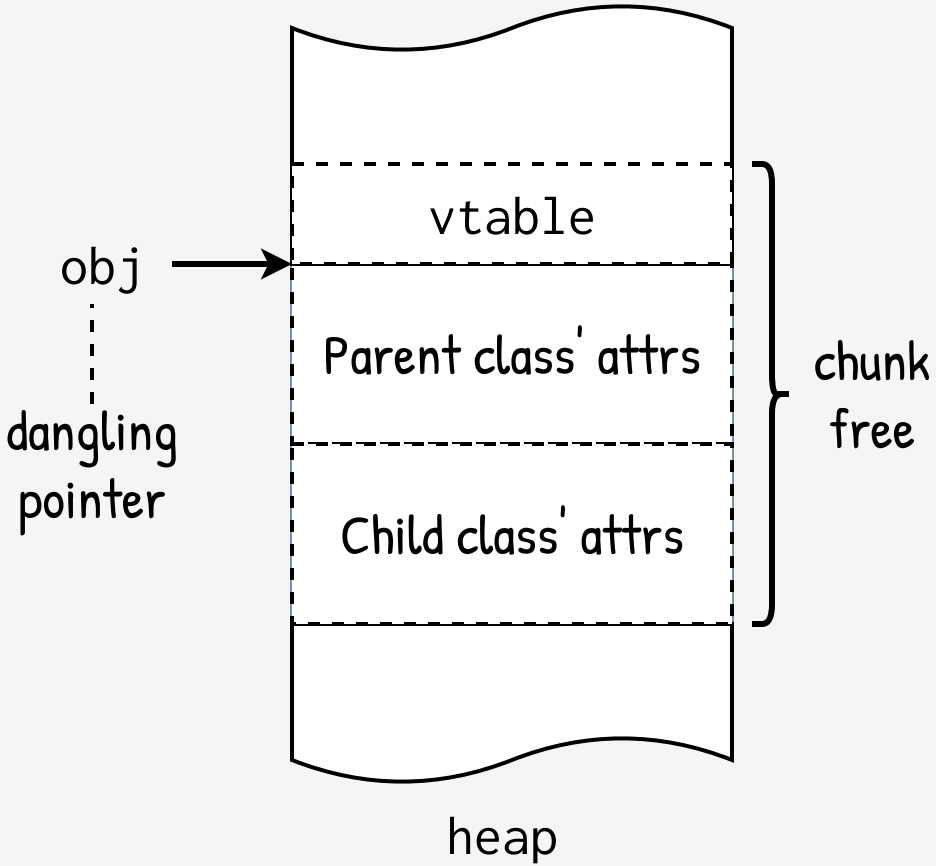

prev sz
cur sz
*prev
*next
malloc(64)
prev sz
cur sz
user ptr
buffer
overflow
prev sz
cur sz
*prev
*next

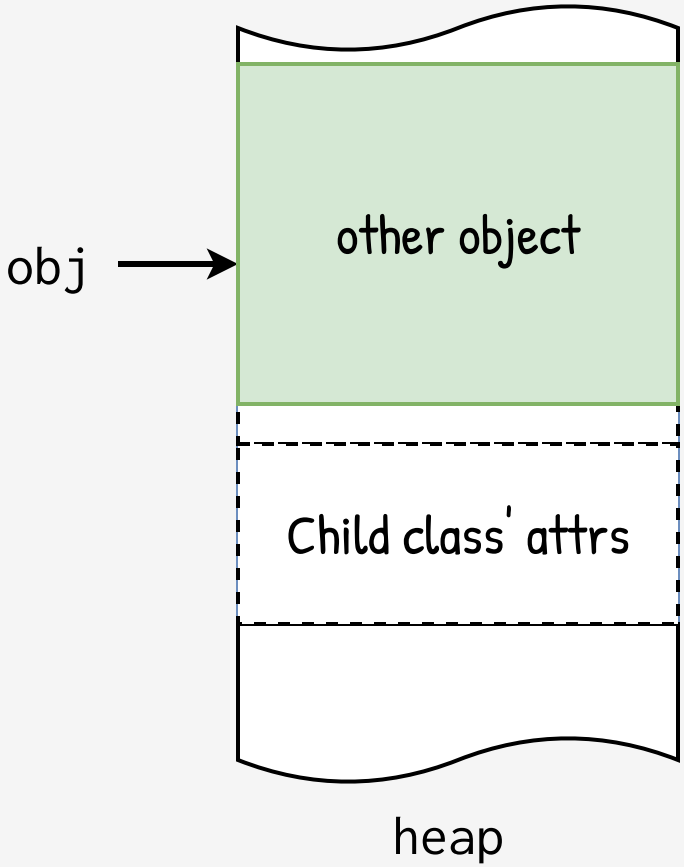

obj
(Parent*)
pointer
Parent class' attrs
Child class' attrs
child
object
heap

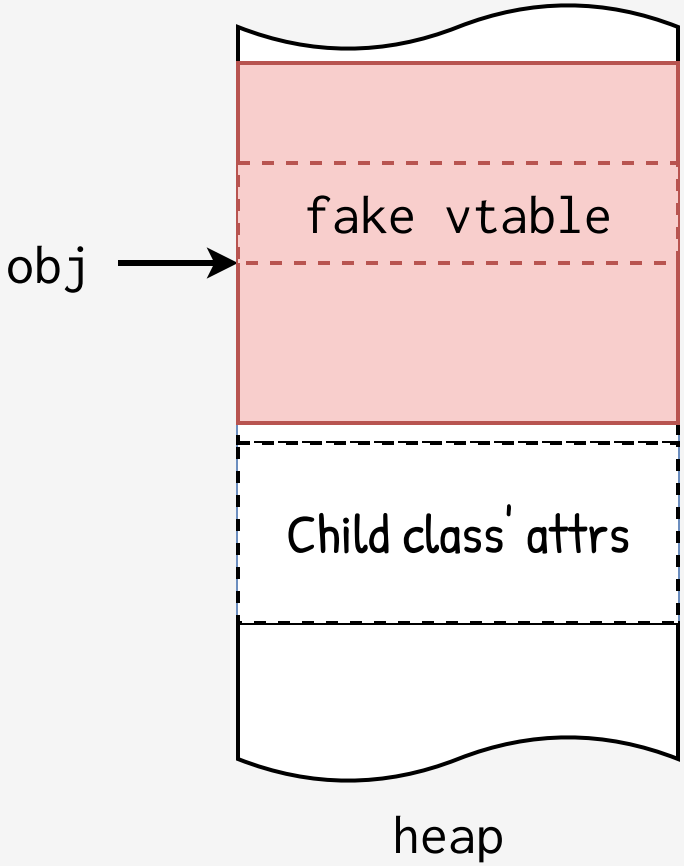

obj
vtable
Parent class' attrs
Child class' attrs
child object
heap

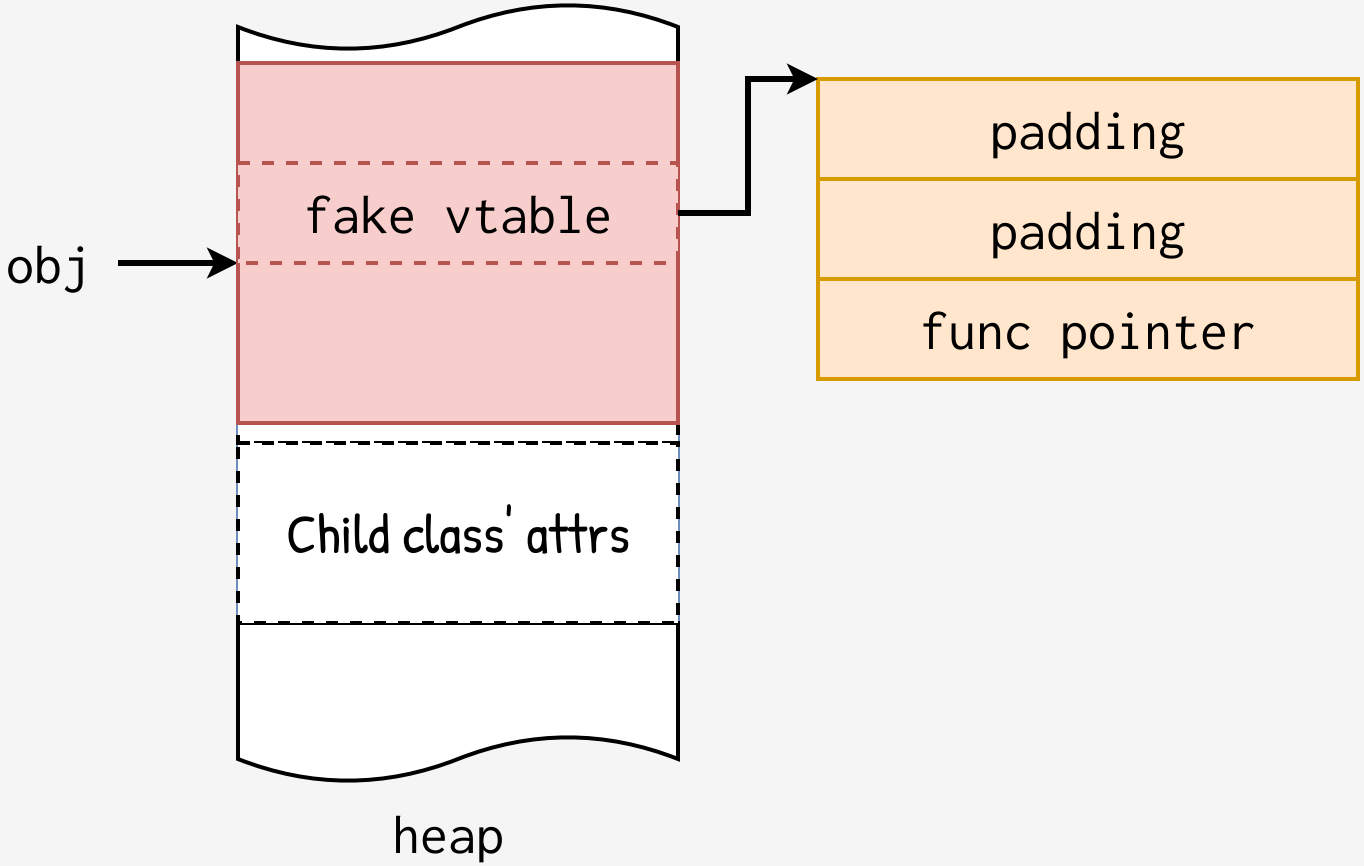

child's func pointers
method 1
method 2
child destructor
vtable
obj
Parent class' attrs
Child class' attrs
heap

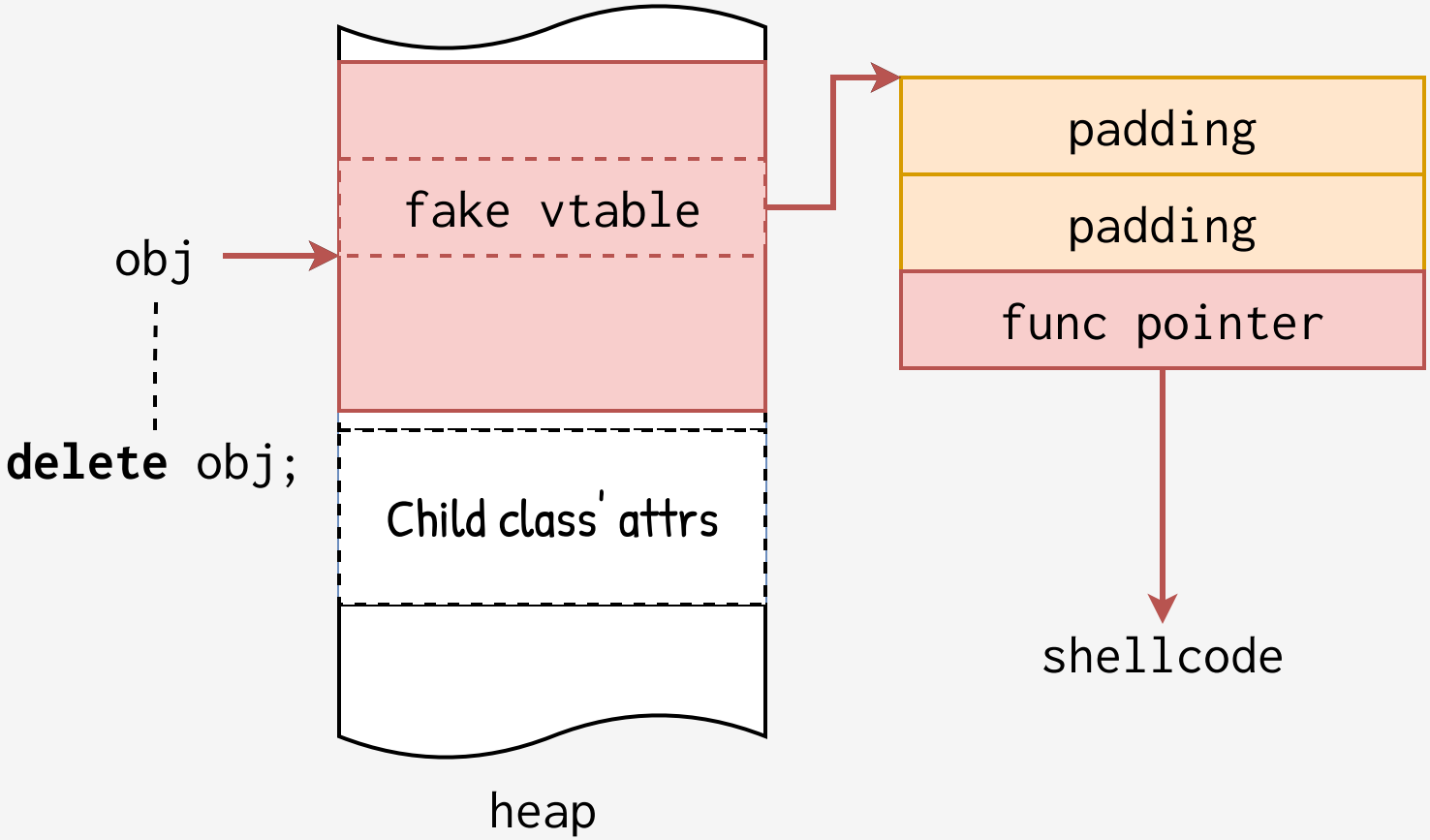

child's func pointers
method 1
method 2
child destructor
vtable
obj
Parent class' attrs
delete obj;
Child class' attrs
heap

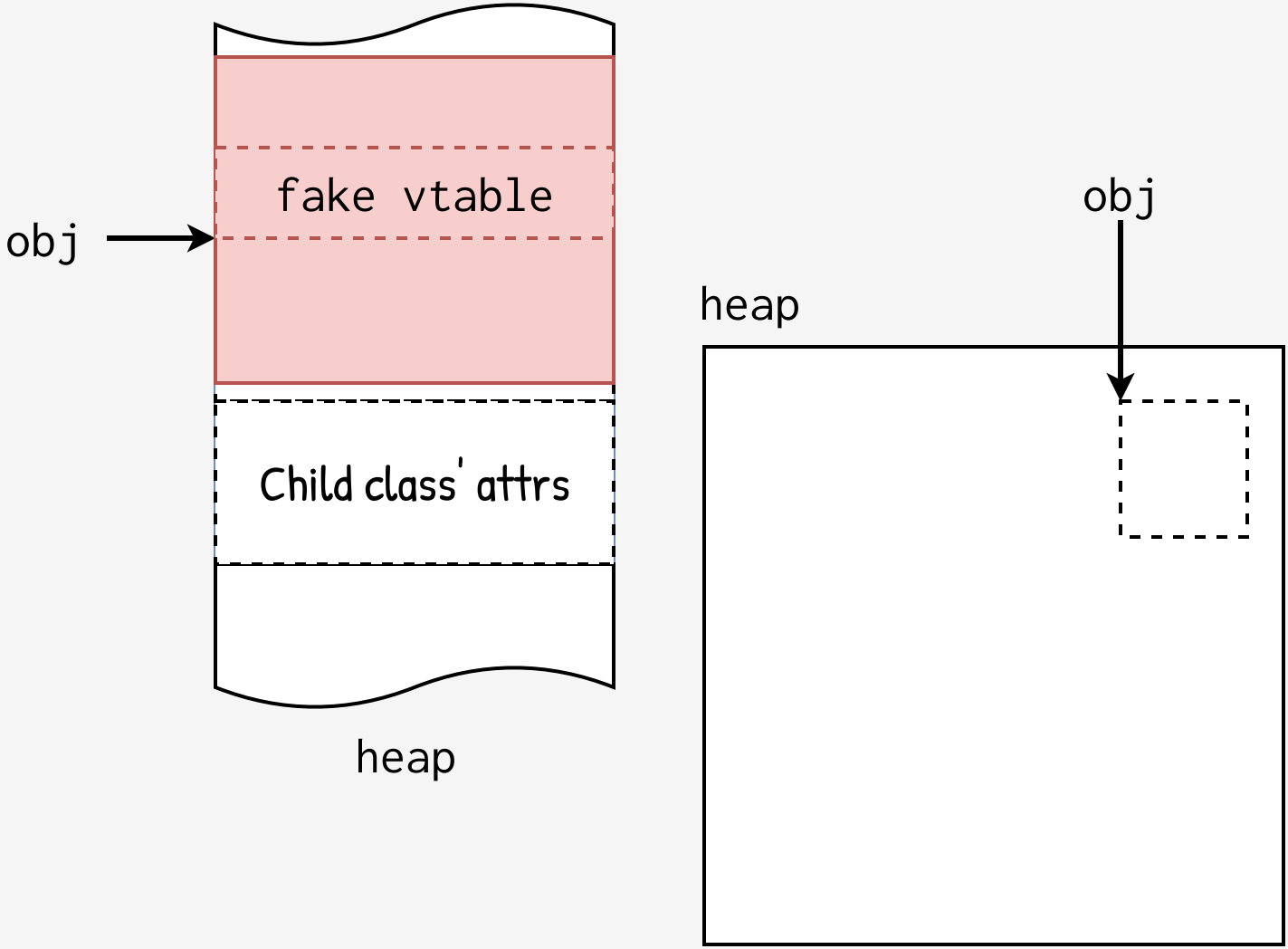

vtable
obj
Parent class' attrs
chunk
free
dangling
pointer
Child class' attrs
heap

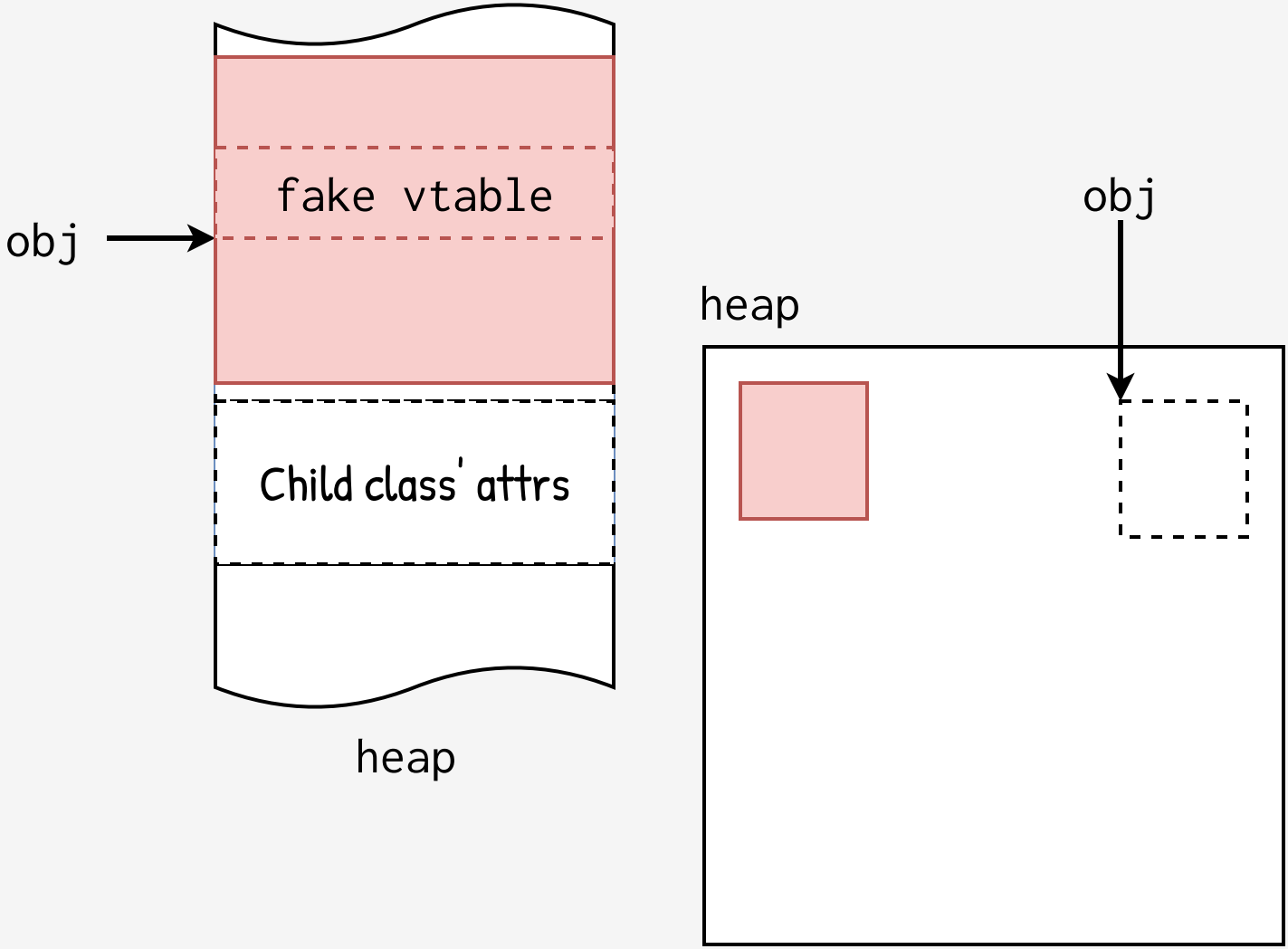

obj
other object
Child class' attrs
heap

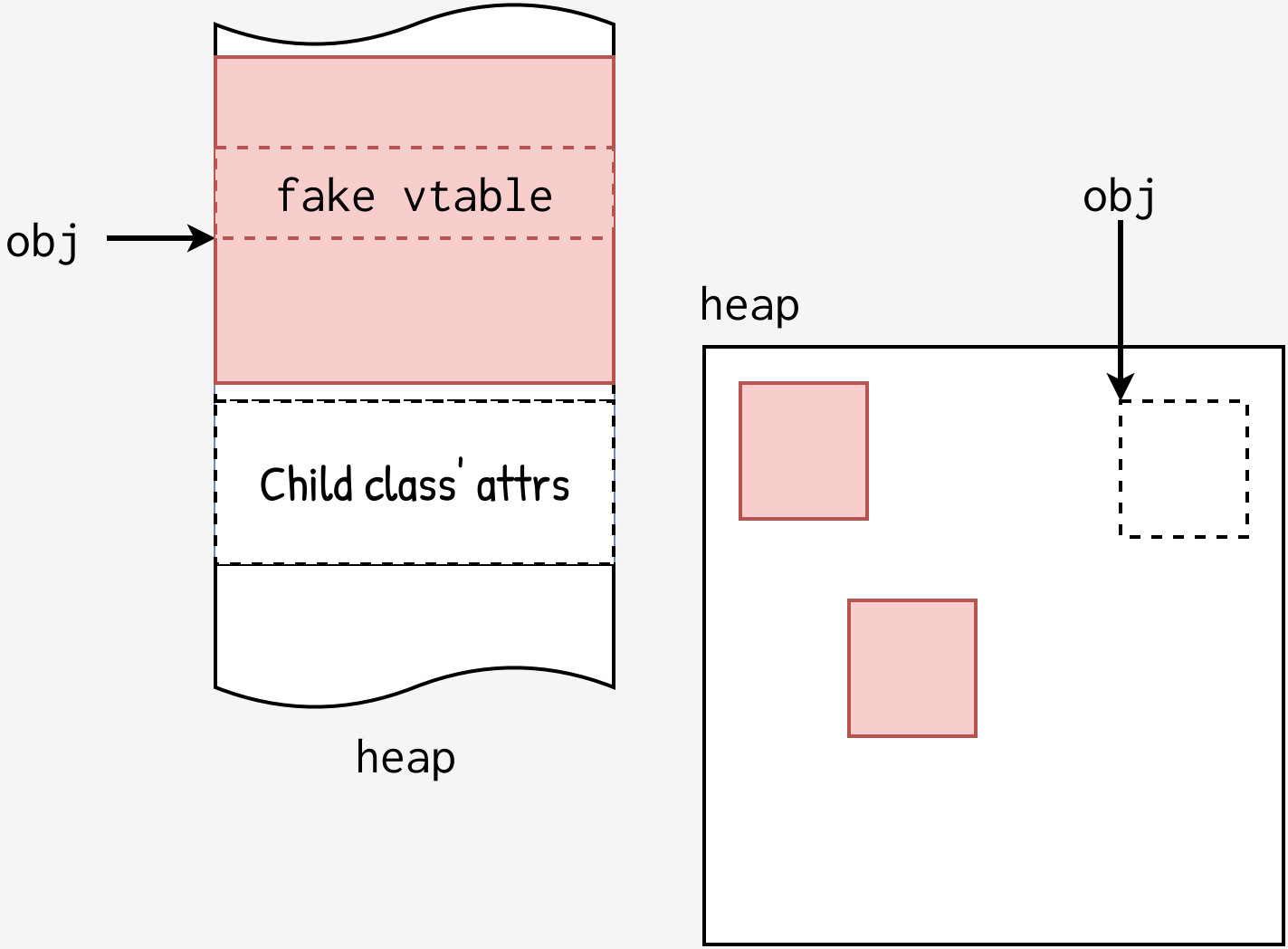

obj
fake vtable
Child class' attrs
heap

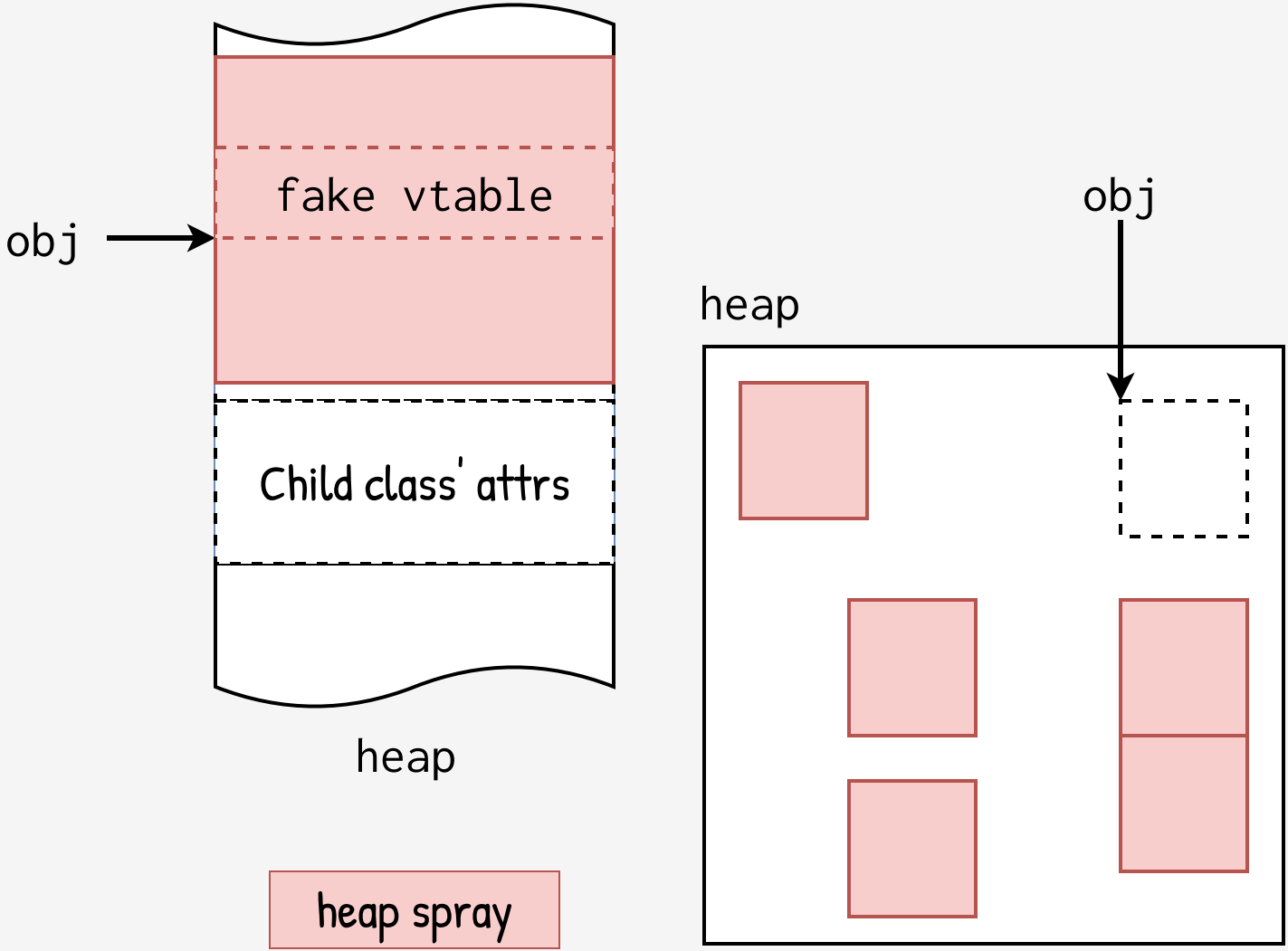

obj
fake vtable
Child class' attrs
heap
padding
padding
func pointer

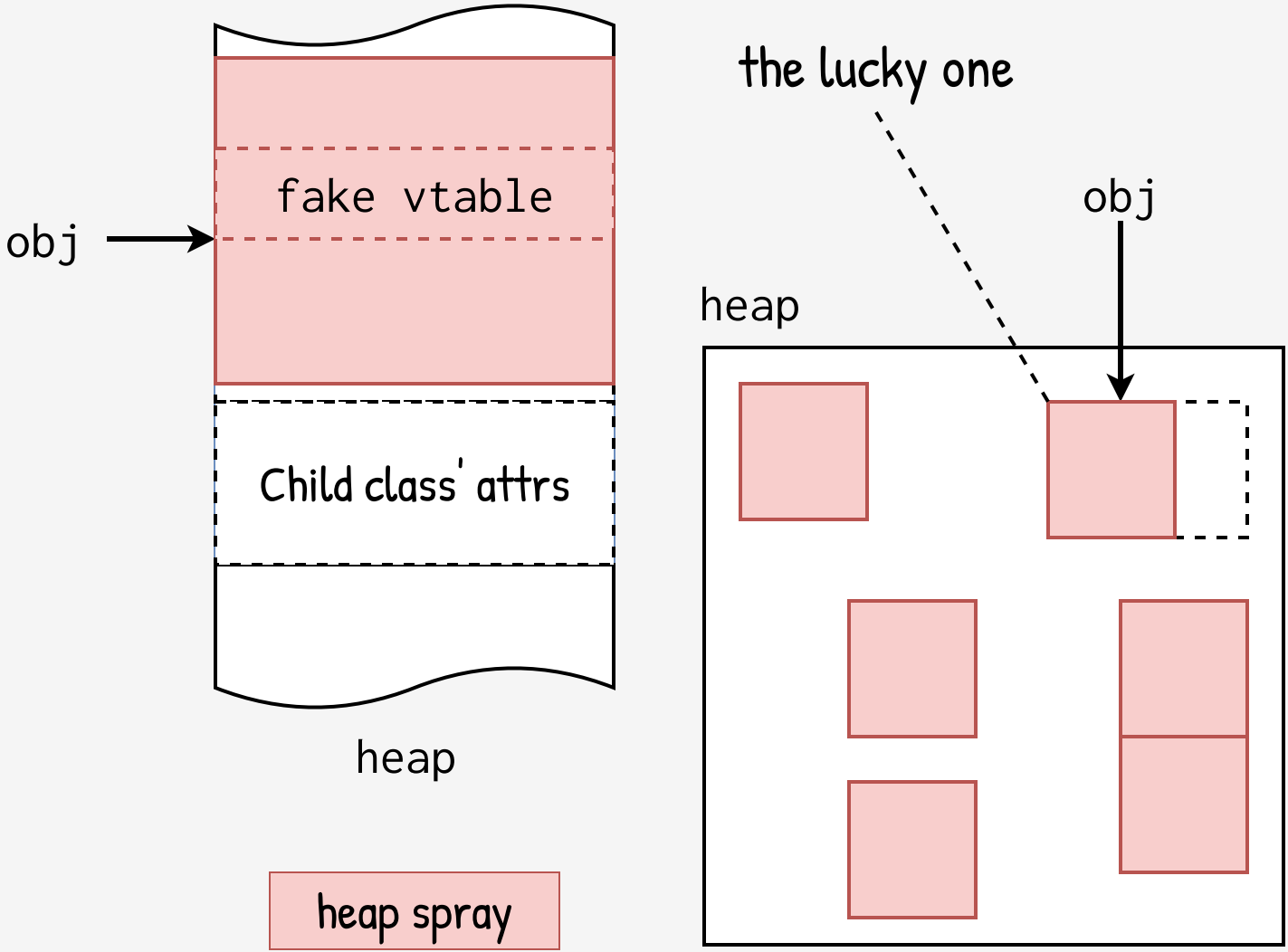

obj
delete obj;
fake vtable
Child class' attrs
heap
padding
padding
func pointer
shellcode

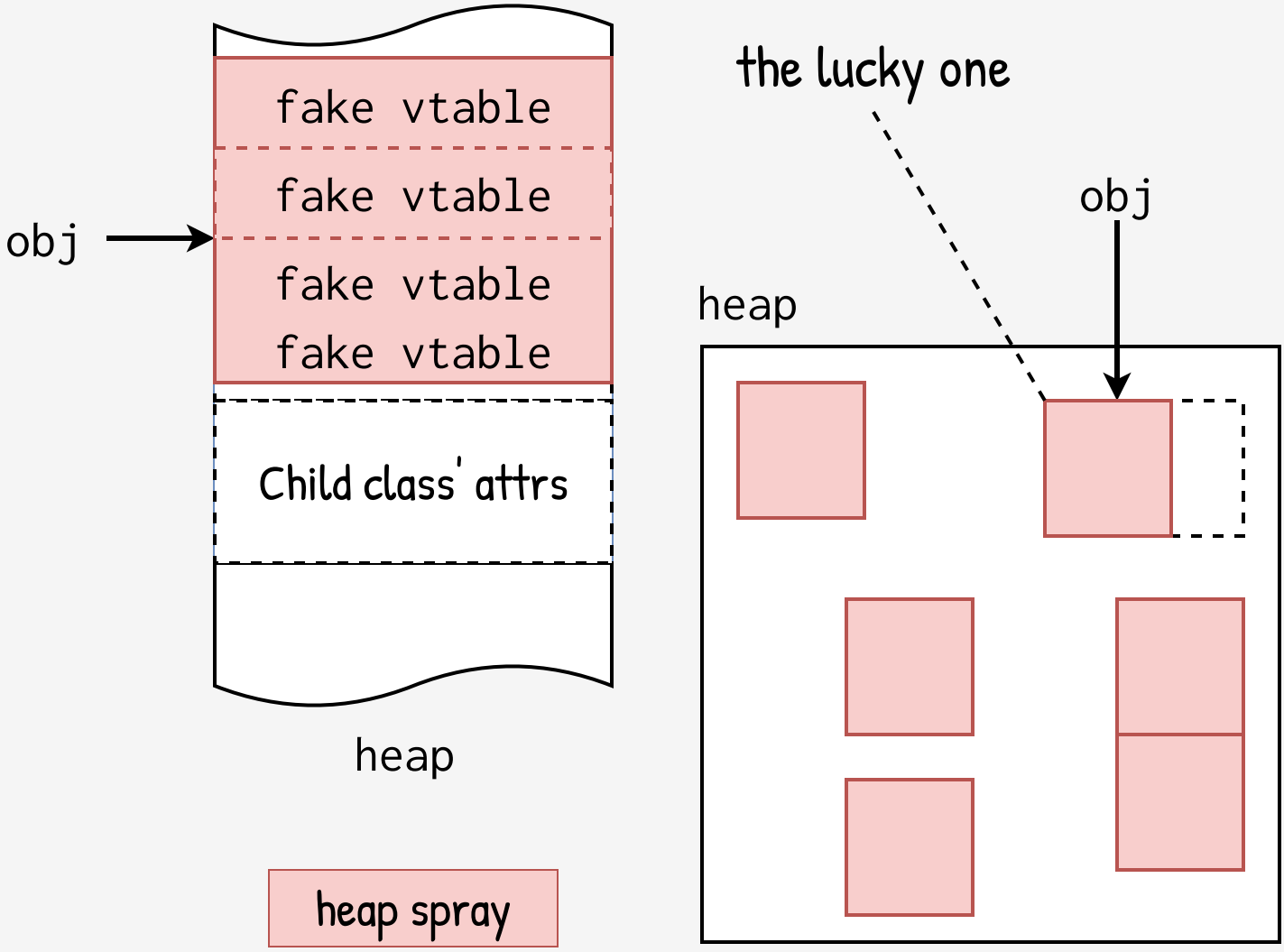

fake vtable
obj
Child class' attrs
heap
obj
heap


obj
fake vtable
Child class' attrs
heap
obj
heap


obj
fake vtable
Child class' attrs
heap
obj
heap


obj
fake vtable
Child class' attrs
heap
heap spray
obj
heap


obj
fake vtable
Child class' attrs
heap
heap spray
the lucky one
obj
heap


fake vtable
fake vtable
fake vtable
fake vtable
obj
Child class' attrs
heap
heap spray
the lucky one
obj
heap